

M17 Teaching Book

10 architecture lessons - self-contained teacher edition

Standalone DE 2026 canonical rebuild - module-level QA candidate

DE17-01 - From requirements to architecture

By the end of this lesson

Turn business, data, SLA, security, recovery and cost constraints into explicit architecture decisions instead of choosing tools first.

1. WATCH / VISUAL FIRST

[DP-700 architecture visual](#) - use 2:19:27 onward as concrete cloud context. [Open current official architecture reference](#). Close the visual before independent work.

2. Teacher mental model

Requirements -> constraints -> options -> decision -> evidence -> revisit

3. Teach it from zero

Architecture is a chain of justified decisions. First collect functional requirements (what the system must do) and nonfunctional requirements (freshness, availability, recovery, scale, security, cost, operability). A product choice without those constraints cannot be defended. Requirements can conflict, so record priority and tradeoffs instead of pretending every pillar can be maximized.

4. Worked architecture reasoning

A retailer needs 30-minute finance freshness, 2-minute inventory alerts, 60-minute RTO and restricted customer PII. Do not start with a product name; split requirements by flow and nonfunctional constraint.

5. Common beginner trap

Choosing a fashionable tool before writing freshness, recovery, scale, security and ownership requirements.

DE17-01 - Decision workshop

Do it with me

- List every consumer and freshness target.
- Separate functional from nonfunctional requirements.
- Circle the three constraints that most strongly change architecture.
- Write one product-neutral architecture decision.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-01 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-02 - Batch vs streaming decisions

By the end of this lesson

Choose batch, streaming or a deliberate hybrid from latency/value/complexity/replay needs.

1. WATCH / VISUAL FIRST

[DP-700 architecture visual](#) - use 2:19:27 onward as concrete cloud context. [Open current official architecture reference](#). Close the visual before independent work.

2. Teacher mental model

Need now? -> event stream; can wait? -> batch; mixed needs -> hybrid with clear truth boundaries

3. Teach it from zero

Batch processes a bounded set on a schedule or trigger. Streaming processes events continuously or in very small increments. The key question is not “can we stream?” but “does the business value of lower latency justify ordering, replay, state, monitoring and operating complexity?” Hybrid designs are normal when consumers need different latency and correctness properties.

4. Worked architecture reasoning

Inventory stockout alerts need <2 minutes while finance needs a reconciled daily close. Use a streaming path for alerts and a controlled batch/serving path for finance.

5. Common beginner trap

Calling streaming “better” without pricing the operational complexity and replay/ordering burden.

DE17-02 - Decision workshop

Do it with me

- Classify each flow as batch/stream/hybrid.
- Name event-time/replay/ordering needs for any stream.
- Name one cheaper batch alternative.
- State what latency change would reverse your choice.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-02 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-03 - Warehouse vs lakehouse

By the end of this lesson

Choose analytical stores from data shape, consumers, governance, SQL/Spark needs and openness rather than slogans.

1. WATCH / VISUAL FIRST

[DP-700 architecture visual](#) - use 2:19:27 onward as concrete cloud context. [Open current official architecture reference](#). Close the visual before independent work.

2. Teacher mental model

Structured SQL concurrency <-> warehouse | mixed files/Spark/open table <-> lakehouse | use both only with clear ownership

3. Teach it from zero

A warehouse emphasizes structured relational analytics, SQL governance and predictable BI serving. A lakehouse combines file/object storage with table semantics and distributed processing for mixed data and engineering/ML. Architecture can use both, but duplicate truth is dangerous: state which layer owns raw truth, curated truth and serving contracts.

4. Worked architecture reasoning

A BI-heavy curated sales mart can suit a warehouse while raw/semi-structured clickstream and Spark transformations suit a lakehouse. Define which layer is authoritative.

5. Common beginner trap

Treating lakehouse and warehouse as mutually exclusive brands instead of workload choices.

DE17-03 - Decision workshop

Do it with me

- State raw, curated and serving truth owners.
- Map data shape and consumer/query pattern.
- Choose warehouse/lakehouse/both and reject one alternative.
- Name one governance risk of duplicated truth.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-03 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-04 - Managed service vs build yourself

By the end of this lesson

Decide where managed services reduce undifferentiated operations and where control/customization justifies ownership.

1. WATCH / VISUAL FIRST

[DP-700 architecture visual](#) - use 2:19:27 onward as concrete cloud context. [Open current official architecture reference](#). Close the visual before independent work.

2. Teacher mental model

Business differentiation? control need? team skill? compliance? ops burden? -> managed/build decision

3. Teach it from zero

Managed services transfer part of patching, availability, upgrades and control-plane operations to a provider. Self-managed systems offer more control but make your team own more failure modes. Compare total operational responsibility, not license price. Build yourself only when the control, portability, compliance or specialization benefit is worth the burden.

4. Worked architecture reasoning

A small team with strict delivery targets should not self-host an orchestrator merely to prove technical ability if a managed option meets requirements and cost.

5. Common beginner trap

Ignoring patching, upgrades, on-call, backups and expertise when comparing “cheap” self-hosted software.

DE17-04 - Decision workshop

Do it with me

- Write team/skill/on-call constraints.
- List what a managed service owns vs your team.
- List a reason self-hosting might be justified.
- Name the exit/lock-in tradeoff.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-04 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-05 - Scaling and bottleneck reasoning

By the end of this lesson

Locate the constrained resource before scaling and choose the dimension that addresses it.

1. WATCH / VISUAL FIRST

[DP-700 architecture visual](#) - use 2:19:27 onward as concrete cloud context. [Open current official architecture reference](#). Close the visual before independent work.

2. Teacher mental model

Measure -> locate bottleneck -> choose scale dimension -> retest -> watch cost/new bottleneck

3. Teach it from zero

Scaling is a response to a measured limit. More workers do not fix every bottleneck. Diagnose whether the limit is CPU, memory, storage I/O, network, upstream quotas, small files, skew, serialization, concurrency or poor query shape. Scale the constrained dimension, rerun the same load, and measure whether the bottleneck moved.

4. Worked architecture reasoning

A pipeline misses SLA because one skewed partition holds 65% of rows. Adding workers may waste money; fix partition/key distribution first, then retest.

5. Common beginner trap

Scaling everything because total volume grew, without identifying CPU, memory, I/O, network, skew, concurrency or upstream limits.

DE17-05 - Decision workshop

Do it with me

- Name a measurable bottleneck signal.
- Choose the scale dimension that addresses it.
- Predict a new bottleneck after scaling.
- Define the validation load you rerun.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-05 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-06 - Reliability and failure domains

By the end of this lesson

Design containment, recovery and validation around explicit failure domains and RTO/RPO.

1. WATCH / VISUAL FIRST

[DP-700 architecture visual](#) - use 2:19:27 onward as concrete cloud context. [Open current official architecture reference](#). Close the visual before independent work.

2. Teacher mental model

Failure domain -> blast radius -> detection -> containment -> recovery -> reconcile

3. Teach it from zero

Reliability means the system can meet agreed availability and recovery goals when components fail. Define failure domains and blast radius. RTO is how long recovery may take; RPO is how much recent state/data the business can lose. Durable raw inputs, checkpoints, idempotent side effects, backups and tested recovery make these goals credible.

4. Worked architecture reasoning

If ingestion succeeds but publish fails, preserve landed raw data and checkpoint truth so recovery can rerun publish without duplicating ingestion side effects.

5. Common beginner trap

Writing “high availability” without naming a failure, recovery target, state source and post-recovery validation.

DE17-06 - Decision workshop

Do it with me

- Name a failure domain and blast radius.
- State RTO/RPO.
- Separate durable state from disposable compute.
- Write recovery and post-recovery reconciliation.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-06 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-07 - Cost-aware architecture

By the end of this lesson

Treat cost as a design constraint and recurring feedback signal, not a cleanup exercise after launch.

1. WATCH / VISUAL FIRST

[DP-700 architecture visual](#) - use 2:19:27 onward as concrete cloud context. [Open current official architecture reference](#). Close the visual before independent work.

2. Teacher mental model

Demand + SLA -> cost drivers -> guardrails -> unit economics -> optimize without breaking requirements

3. Teach it from zero

Cost is shaped by compute time, storage volume/retention, data movement, always-on capacity, concurrency and managed-service pricing. Cost optimization is not “use the smallest machine.” It is meeting requirements with controlled spend. Record unit-cost drivers and guardrails, then revisit them when workload volume or SLA changes.

4. Worked architecture reasoning

A batch job can finish in 20 minutes on a large cluster or 55 minutes on a smaller one. If SLA is 90 minutes, the cheaper configuration may be the better architecture.

5. Common beginner trap

Equating cheapest with best, or paying for peak capacity 24/7 when workload timing allows elasticity.

DE17-07 - Decision workshop

Do it with me

- List top three cost drivers.
- Set two guardrails.
- Identify one performance/cost tradeoff.
- State a growth threshold that forces redesign.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-07 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-08 - Security and governance by design

By the end of this lesson

Place identity, least privilege, data classification, lineage, retention and audit boundaries into the architecture before implementation.

1. WATCH / VISUAL FIRST

[DP-700 architecture visual](#) - use 2:19:27 onward as concrete cloud context. [Open current official architecture reference](#). Close the visual before independent work.

2. Teacher mental model

Classify -> identity -> authorize -> encrypt/mask -> trace lineage -> retain/audit -> review

3. Teach it from zero

Security and governance follow data through the system. Classify sensitive fields, use identities rather than shared credentials, grant least privilege, protect data in transit/at rest, mask or aggregate for broader consumption, record lineage and retention, and keep audit evidence. A diagram should show trust boundaries and ownership.

4. Worked architecture reasoning

Raw customer email stays restricted; curated regional aggregates can be broadly consumed. The diagram should show where sensitivity decreases and who can cross each boundary.

5. Common beginner trap

Adding a security box to the final diagram without connecting permissions and data classification to real flows.

DE17-08 - Decision workshop

Do it with me

- Classify two sensitive fields.
- Name identities and access scopes.
- Show where masking/aggregation occurs.
- Add lineage/retention/audit evidence.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-08 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-09 - Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes

By the end of this lesson

Explain what each advanced concept solves, one risk/tradeoff, and when NOT to use it; awareness/translation only, not five new mandatory platforms.

1. WATCH / VISUAL FIRST

Open the bundled promotion-foundations HTML. Watch/read only the mapped CDC, contracts, mesh, Vault and Kubernetes sections. Then close it.

2. Teacher mental model

CDC | contracts | Vault | data mesh | Kubernetes -> problem solved + operational cost + use/no-use trigger

3. Teach it from zero

CDC captures source changes so downstream systems can react without repeatedly scanning entire tables. Data contracts make producer/consumer expectations explicit. Vault manages secret lifecycle and controlled retrieval. Data mesh is an organizational/data-product model with domain ownership, not software. Kubernetes orchestrates containerized workloads. Each solves a real class of problem and each adds operational cost.

4. Worked architecture reasoning

A mature platform may use CDC for low-latency database changes, contracts for producer/consumer compatibility, Vault for secret lifecycle, mesh for domain ownership, and Kubernetes for container orchestration - but a small batch team may need none of them yet.

5. Common beginner trap

Name-dropping promotion technologies without being able to reject them when complexity exceeds the business need.

DE17-09 - Decision workshop

Do it with me

- For each concept: problem solved.
- For each: one failure/operational burden.
- For each: a use trigger.
- For each: a reject trigger.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-09 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?

DE17-10 - Architecture review and defense

By the end of this lesson

Defend one end-to-end architecture from requirements through tradeoffs, failure recovery, security, cost and scaling without tutorial help.

1. VISUAL POLICY

No tutorial by design. Architecture defense is an independent skill. Keep videos, Review and AI solving closed for the attempt.

2. Teacher mental model

Requirement -> decision -> rejected alternative -> risk -> evidence -> changed constraint -> revised decision

3. Teach it from zero

Architecture review is a technical argument, not a vocabulary quiz. Start with measurable requirements. Walk one data flow end to end. For each major choice state why, what alternative you rejected, what can fail, how you detect/recover, and what you validate afterward. Then accept a changed constraint and show which decision moves.

4. Worked architecture reasoning

No worked answer by design. You receive a changed scenario and must defend the architecture aloud and in an ADR.

5. Common beginner trap

Presenting a pretty diagram with no measurable requirements, rejected alternative, failure path or validation plan.

DE17-10 - Decision workshop

Do it with me

- Defend one architecture in 5 minutes.
- Answer why-not-the-alternative for three choices.
- Walk one failure through recovery.
- Accept one changed constraint and revise one decision.

Expected proof

One saved architecture/ADR artifact with measurable requirements, at least one rejected alternative, a named risk/failure, and a validation statement. A diagram alone is not proof.

Your turn - do not copy

Close this page and complete the matching DE17-10 Independent Practice task. Use your own architecture language. Save the attempt before opening Review.

STOP - explain aloud

What requirement drove your decision? What did you reject? What can fail? What evidence would make you change the decision?